

Shiki ChatGPT Light 渲染测试

Shiki + ChatGPT Light PDF 渲染测试

这份文档用于验证当前 PDF 项目的新渲染链路：默认主题为 `chatgpt-light`，代码高亮由 Shiki 生成，页面背景和打印页边背景均为纯白。这里额外增加一段说明，避免内容校验过短：本测试覆盖数学公式、TypeScript、Python、Bash、JSON、CSS、表格和列表，主要观察白底页面、浅色代码块、KaTeX 公式排版和 PDF 打印页边背景是否一致。

1. 公式渲染

行内公式示例：当 $a > 0$ 时，二次函数 $f(x) = ax^2 + bx + c$ 的顶点横坐标为 $x = -\frac{b}{2a}$ 。

块级公式示例：

$$\int_0^1 x^2 dx = \left. \frac{x^3}{3} \right|_0^1 = \frac{1}{3}.$$

矩阵公式示例：

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{pmatrix}, \quad \det(A) = 1.$$

2. TypeScript 高亮

```
interface PdfBuildJob {
  id: string;
  theme: 'chatgpt-light' | string;
  input: string;
  output: string;
}

function describeJob(job: PdfBuildJob): string {
  const file = job.output.split('/').at(-1) ?? job.output;
  return `${job.id} -> ${file} using ${job.theme}`;
}

const job: PdfBuildJob = {
  id: 'shiki-chatgpt-light-render-test',
  theme: 'chatgpt-light',
  input: 'notes.md',
  output: 'dist/test.pdf'
};

console.log(describeJob(job));
```

3. Python 高亮

```
from dataclasses import dataclass
from pathlib import Path

@dataclass
class RenderResult:
    html: Path
    pdf: Path
    ok: bool = True

def normalize_title(title: str) -> str:
    return "-".join(title.strip().lower().split())

result = RenderResult(
    html=Path("dist/test.html"),
    pdf=Path("dist/test.pdf"),
)

print(normalize_title("Shiki ChatGPT Light"), result.ok)
```

4. Bash / JSON / CSS 高亮

```
npm install
PDF_THEME=chatgpt-light SHIKI_THEME=github-light npm run build:pdf
```

```
{
  "theme": "chatgpt-light",
  "codeHighlighter": "shiki",
  "shikiTheme": "github-light",
  "pageBackground": "#ffffff"
}
```

```
@page {
  background: #ffffff;
}

.markdown-rendered {
  background: #ffffff !important;
  color: #0d0d0d !important;
}
```

5. 表格与列表

项目	当前选择	说明
Markdown	markdown-it	继续保留原链路
公式	KaTeX	Node 端预渲染
代码高亮	Shiki	使用 <code>github-light</code>
PDF	Chromium	Puppeteer 打印

- 页面背景应为纯白。
- 页边背景也应为纯白。
- 代码块应为浅灰背景，适合白底阅读。
- 公式应保持 KaTeX 原生字体，不被主题 CSS 破坏。